



Yeditepe University

CSE224: Introduction to Digital Systems

Term Project Handout

Design, Implementation, and Verification of a VSCPU Processor

Prepared by: **Asst. Prof. Gizem Süngü Terci**

1. Objective

The purpose of this term project is to design, implement, verify, and synthesize a fully functional **VSCPU (Very Simple CPU)** using Verilog HDL. Your processor must support the entire instruction set architecture (ISA), which is available in your lecture notes, execute a given assembly program correctly, and be extended with two new instructions: SUB and SUBi.

2. Project Steps

The project consists of four major components, each of which must be demonstrated and documented.

2.1 Step 1 — Complete the VSCPU Verilog Design

You are required to implement all instructions in the VSCPU ISA, including:

- Arithmetic Instructions: ADD, ADDi, MUL, MULi
- Logical Instructions: NAND, NANDi
- Shift Instructions: SRL, SRLi
- Comparison Instructions: LT, LTi
- Copy Instructions: CP, CPi, CPI, CPIi
- Branch Instructions: BZJ, BZJi

We have already implemented **ADDi** and **CP** instructions and the Verilog implementation of the general design of **VSCPU** and these instructions are available on YULearn. The Verilog implementation of all instructions must follow the microoperation descriptions provided in the ISA.

2.2 Step 2 — Execute and Verify the VSCPU Program

You must load the following assembly program into your memory module and verify execution **after every instruction** by comparing your waveform outputs with the reference simulation on the `cpu.tc` website (check your lecture notes for how to use `cpu.tc`).

```
0: CPi 110 3
1: ADD 100 101
2: MUL 100 102
3: SRLi 102 1
4: CP 104 100
5: ADDi 104 5
6: NAND 104 108
7: NANDi 104 5
8: SRL 108 102
9: MULi 108 3
10: ADD 110 103
11: CP 112 110
12: LT 112 111
13: BZJ 111 112
14: BZJi 101 11
19: MULi 101 3
20: CP 105 102
21: LTi 105 2
22: BZJ 113 105
35: BZJi 111 53
54: CPIi 114 111
55: CPI 121 102
100:5
101:8
102:16
103:4294967295
108:65543
111:1
113:35
114:120
```

Your report must include waveform screenshots for the instructions listed in Step 4.

2.3 Step 3 — Synthesize Your VSCPU

Using Vivado or Synopsys, synthesize your VSCPU and report the synthesis results, including LUT/ALM usage, flip-flop usage, timing information, and area utilization.

2.4 Step 4 — Prepare Final Report

Your project report must include:

- Full datapath and control signal explanation
- Step-by-step simulation screenshots and verification
- Description of the implementation of all instructions
- Synthesis results and interpretation
- C program corresponding to the test assembly code

2.4.1 Step-by-step simulation screenshots and verification

Since the full instruction sequence is long, only the following steps must be verified with screenshots in your report. For each instruction shown below, demonstrate that your Verilog design produces the same output as the VSCPU simulator.

1. SRL Verification in Figure 1 (Displaying only the updated values of mem[108] and mem[102] at the relevant PC after execution is sufficient).

```
*****
current_instruction: SRL 108 102
program counter    : 8
Memory content before executing instruction
mem[ 108 ] : 65543
mem[ 102 ] : 8
Memory content after executing instruction
mem[ 108 ] : 256
mem[ 102 ] : 8
*****
```

Figure 1: SRL instruction

2. SRLi Verification in Figure 2
3. ADD Verification in Figure 3.
4. ADDi Verification in Figure 4.
5. CP Verification in Figure 5.

```

* * * * *
current_instruction: SRLi 102 1
program counter    : 3
Memory content before executing instruction
mem[ 102 ]        : 4
Memory content after executing instruction
mem[ 102 ]        : 2
* * * * *

```

Figure 2: SRLi instruction

```

* * * * *
current_instruction: ADD 100 101
program counter    : 1
Memory content before executing instruction
mem[ 100 ]        : 208
mem[ 101 ]        : 8
Memory content after executing instruction
mem[ 100 ]        : 216
mem[ 101 ]        : 8
* * * * *

```

Figure 3: ADD instruction

```

* * * * *
current_instruction: ADDi 104 5
program counter    : 5
Memory content before executing instruction
mem[ 104 ]        : 1728
Memory content after executing instruction
mem[ 104 ]        : 1733
* * * * *

```

Figure 4: ADDi instruction

```

* * * * *
current_instruction: CP 105 102
program counter    : 20
Memory content before executing instruction
mem[ 105 ]        : 0
mem[ 102 ]        : 2
Memory content after executing instruction
mem[ 105 ]        : 2
mem[ 102 ]        : 2
* * * * *

```

Figure 5: CP instruction

```

run all mode selected
* * * * *
current_instruction: CPi 110 3
program counter    : 0
Memory content before executing instruction
mem[ 110 ]        : 0
Memory content after executing instruction
mem[ 110 ]        : 3
* * * * *
* * * * *

```

Figure 6: CPi instruction

6. CPi Verification in Figure 6.

7. LTi Verification in Figure 7.

```

* * * * *
current_instruction: LTi 105 2
program counter    : 21
Memory content before executing instruction
mem[ 105 ]        : 2
Memory content after executing instruction
mem[ 105 ]        : 0
* * * * *

```

Figure 7: LTi instruction

8. MULi Verification in Figure 8.

```

* * * * *
current_instruction: MULi 101 3
program counter    : 19
Memory content before executing instruction
mem[ 101 ]        : 8
Memory content after executing instruction
mem[ 101 ]        : 24
* * * * *

```

Figure 8: MULi instruction

9. MUL Verification in Figure 9.

```

* * * * *
current_instruction: MUL 100 102
program counter    : 2
Memory content before executing instruction
mem[ 100 ]        : 216
mem[ 102 ]        : 8
Memory content after executing instruction
mem[ 100 ]        : 1728
mem[ 102 ]        : 8
* * * * *

```

Figure 9: MUL instruction

10. BZJi, BZJ, CPIi Verification in Figure 10
11. LT Verification in Figure 11
12. NAND and NANDi Verification in Figure 12

```

* * * * *
current_instruction: BZJ 113 105
program counter    : 22
mem[ 113 ] : 35
mem[ 105 ] : 0
* * * * *

* * * * *
current_instruction: BZJi 111 53
program counter    : 35
mem[ 111 ] : 1
* * * * *

* * * * *
current_instruction: CPIi 114 111
program counter    : 54
Memory content before executing instruction
mem[ 114 ] : 120
mem[ 111 ] : 1
mem[ 120 ] : 0
Memory content after executing instruction
mem[ 114 ] : 120
mem[ 111 ] : 1
mem[ 120 ] : 1
* * * * *

```

Figure 10: BZJ, BZJi and CPIi instructions

```

* * * * *
current_instruction: LT 112 111
program counter    : 12
Memory content before executing instruction
mem[ 112 ] : 2
mem[ 111 ] : 1
Memory content after executing instruction
mem[ 112 ] : 0
mem[ 111 ] : 1
* * * * *

```

Figure 11: LT instruction

```

* * * * *
current_instruction: NAND 104 108
program counter    : 6
Memory content before executing instruction
mem[ 104 ] : 1733
mem[ 108 ] : 768
Memory content after executing instruction
mem[ 104 ] : 4294966783
mem[ 108 ] : 768
* * * * *

* * * * *
current_instruction: NANDi 104 5
program counter    : 7
Memory content before executing instruction
mem[ 104 ] : 4294966783
Memory content after executing instruction
mem[ 104 ] : 4294967290
* * * * *

```

Figure 12: NAND and NANDi instructions

3. New Instructions: SUB and SUBi

You must add two new instructions to the VSCPU design.

3.1 SUB Instruction

Opcode: 3'b000, im = 1'b0, IW[13] = 1'b1

Microoperations:

```
R1 <- mem[IW[27:14]]
R2 <- mem[IW[13:0]]
R2 <- ~R2
mem[IW[27:14]] <- (R1 - R2)
PC <- PC + 1
```

3.2 SUBi Instruction

Opcode: 3'b000, im = 1'b1, IW[13] = 1'b1

Microoperations:

```
R1 <- mem[IW[27:14]]
R2 <- IW[13:0]
R2 <- ~R2
mem[IW[27:14]] <- (R1 - R2)
PC <- PC + 1
```

You must implement these new instructions in your Verilog design.

4. Submission Requirements

You must submit the following:

1. **Final Report (PDF)** including EDAPLAYGROUND project link
2. **VSCPU Verilog design files**
3. **Testbench code**

5. Grading

Component	Weight
Correct VSCPU Design	35%
Instruction Implementation (incl. SUB/SUBi)	20%
Simulation & Verification	20%
Synthesis Results	10%
Report Quality	15%